```
 # FPL-Oracle System Architecture (Current Progress – Pre-Cook)

**Version:** 0.3 – “Pre-Cook Pipeline”
**Scope:** End-to-end architecture from data sources → Redis state → normalized schemas.
**Status:** Producers + Pantry + Schemas implemented. Cooks and above: planned.

---

## 1. High-level pipeline

FPL-Oracle is a distributed prediction system for Fantasy Premier League (FPL) built around a layered pipeline:

**Data Sources → Producers → Redis Pantry → Cooks → Aggregator → Manager → API/UI**

This document captures the system **up to and including the Redis Pantry and data schemas**, i.e. everything that exists **before** analytical workers (“Cooks”).

---

## 2. Codebase layout (current)

```text
fpl_oracle/
│
├── config/                # Global settings & data schemas
│   ├── data_maps.py
│   ├── data_struct.py
│   └── settings.py
│
├── producer/              # Distributed scrapers (data ingest)
│   ├── producer.py
│   ├── fotmob_scrapper.py
│   ├── fpl_scraper.py
│   └── reddit_scraper.py
│
├── db/                    # Redis abstraction layer (Pantry)
│   ├── db_redis.py
│   ├── players.py
│   └── teams.py
│
├── cook/                  # (Present but out of scope in this doc)
│   ├── cook.py
│   ├── fixture_cook.py
│   ├── form_cook.py
│   └── minute_cook.py
│
├── utils/                 # Shared utilities
│   ├── data_to_txt.py
│   ├── log.py
│   ├── pw_browser_async.py
│   └── scraper.py
│
└── main.py                # Orchestration entry point
```

This document focuses on: `producer/`, `db/`, `config/` and the **data structures**.

---

3. Producers (data ingestion layer)

3.1 Goals

- Fetch raw data from multiple external sources.
- Normalize into internal schemas (PLAYERS, TEAMS, FIXTURES, FPL_FIXTURES).
- Write to Redis using stable keys (`player:{id}`, `team:{id}`, `fixture:{id}`, `gw:{id}`).

3.2 Components

- `fpl_scraper.py`
  - Source: Official FPL API.
  - Writes: `PLAYERS`, `TEAMS`, `FPL_FIXTURES` base fields.
  - Data: prices, minutes, status, ICT, ranks, ep_next/ep_this, GW metadata.
- `fotmob_scrapper.py`
  - Source: FotMob (HTML/JSON).
  - Writes: `TEAMS.expected`, `TEAMS.strength`, `PLAYERS.expected`, some per-90 stats.
  - Data: xG, xGA, xGI, xGC, team xG/xGA/xPTS, strength coefficients.
```

- `reddit_scraper.py`
  - Source: Reddit/X (news, rumors).
  - Writes: (future) injury/rotation signals, qualitative flags.
  - Data: injury rumors, leaked lineups, tactical notes (to be consumed by future Cooks).
- `producer.py`
  - Orchestrates async runs of all scrapers.
  - Ensures periodic refresh and error isolation per source.

3.3 Design principles

- Producers **never** depend on each other.
- Each producer only updates its own subset of fields in Redis.
- Failures in one producer do not corrupt shared state.

---

4. Redis Pantry (state store)

4.1 Role

Redis is the **central state store** for all normalized football and FPL data:

- Keyed by stable IDs (player, team, fixture, gameweek).
- Updated incrementally by producers.
- Read later by Cooks, Aggregator, Manager, API.

4.2 Access layer

- `db_redis.py`
  - Connection management, serialization/deserialization.
  - Helper methods for `get_player`, `set_player`, `get_team`, `set_team`, `get_fixture`, etc.
- `players.py` / `teams.py`
  - Encapsulate schema-aware access to PLAYERS and TEAMS structures.
  - Provide typed getters/setters and convenience methods (e.g. `get_team_strength`, `get_player_form`).

---

5. Canonical data structures

All producers write into a **shared canonical schema** defined in `config/data_struct.py`.
Below are the current structures.

5.1 Gameweek metadata: `FPL_FIXTURES`

```
FPL_FIXTURES = {
   "id": 0,
   "name": "",
   "deadline_time": "",
   "is_current": "bool",
   "is_previous": "bool",
   "is_next": "bool",
   "finished": "bool",
   "data_checked": "bool",
   "highest_score": 0,
   "most_selected": 0,
   "most_transferred_in": 0,
   "top_element": 0,
   "top_element_info": {
       "id": 0,
       "points": 0,
   },
   "most_captained": 0,
   "most_vice_captained": 0,
}
```

Purpose:

- Represents a single gameweek (event) in FPL.
- Used for: current GW detection, deadline awareness, crowd wisdom (most captained/selected), and top performer metadata.

5.2 Match-level fixtures: `FIXTURES`

```
FIXTURES = {
    "event": 0,
    "finished": "bool",
    "id": 0,
    "kickoff_time": "2025-08-15T19:00:00Z",
    "started": "true",
    "team_a": "4",
    "team_a_score": "2",
    "team_h": "12",
    "team_h_score": "4",
    "stats": {
        "goals_scored": {"a": [], "h": []},
        "assists": {"a": [], "h": []},
        "own_goals": {"a": [], "h": []},
        "penalties_saved": {"a": [], "h": []},
        "penalties_missed": {"a": [], "h": []},
        "yellow_cards": {"a": [], "h": []},
        "red_cards": {"a": [], "h": []},
        "saves": {"a": [], "h": []},
        "bonus": {"a": [], "h": []},
        "bps": {"a": [], "h": []},
        "defensive_contribution": {"a": [], "h": []},
    },
}
```

Each `stats[identifier]["a" or "h"]` entry is a list of:

```
{ "element": <player_id>, "value": <int> }
```

Purpose:

- Represents a single match between two teams in a given gameweek.
- Used for: form calculation, per-90 stats, role detection (penalties, bonus magnets), and model validation.

5.3 Team-level data: `TEAMS`

```
TEAMS = {
    "name": "",
    "short_name": "",
    "tid": 0,
    "table": {
        "win": 0,
        "draw": 0,
        "loss": 0,
        "played": 0,
        "points": 0,
        "position": 0,
        "goals": 0,
        "conceded": 0,
        "form": [],
        "next": "name",
    },
    "expected": {
        "xg": 0,
        "xg_difference": 0,
        "xga": 0,
        "xga_difference": 0,
        "xpts": 0,
        "xpts_difference": 0,
    },
    "strength": {
        "strength": 0,
        "strength_overall_home": 0,
        "strength_overall_away": 0,
        "strength_attack_home": 0,
        "strength_attack_away": 0,
        "strength_defence_home": 0,
        "strength_defence_away": 0,
    },
}
```

Purpose:

- Encodes both **actual table performance** and **underlying expected metrics**.

- Used for: team strength modelling, fixture difficulty, match strength, and opponent adjustment in xP.

5.4 Player-level data: PLAYERS

```
PLAYERS = {
   "id": "",
   "team_code": "",
   "now_cost": 0,
   "web_name": "",
   "total_points": 0,
   "status": "",
   "form": "",
   "element_type": "",
   "stats": {
       "minutes": 0,
       "goals_scored": 0,
       "assists": 0,
       "clean_sheets": 0,
       "goals_conceded": 0,
       "own_goals": 0,
       "penalties_saved": 0,
       "penalties_missed": 0,
       "yellow_cards": 0,
       "red_cards": 0,
       "recoveries": 0,
       "tackles": 0,
       "saves": 0,
       "clearances_blocks_interceptions": 0,
       "defensive_contribution": 0,
       "starts": 0,
       "influence": 0,
       "creativity": 0,
       "threat": 0,
       "corners_and_indirect_freekicks_order": "",
       "direct_freekicks_order": "",
       "penalties_order": "",
   },
   "fpl_stats": {
       "value_form": 0,
       "value_season": 0,
       "bonus": 0,
       "bps": 0,
       "ict_index": 0,
       "event_points": 0,
       "in_dreamteam": 0,
       "dreamteam_count": 0,
       "selected_by_percent": 0,
       "transfers_in": 0,
       "transfers_in_event": 0,
       "transfers_out": 0,
       "transfers_out_event": 0,
   },
   "rank": {
       "selected_rank": 0,
       "selected_rank_type": "",
       "influence_rank": 0,
       "influence_rank_type": "",
       "creativity_rank": 0,
       "creativity_rank_type": "",
       "threat_rank": 0,
       "threat_rank_type": "",
       "ict_index_rank": 0,
       "ict_index_rank_type": "",
       "now_cost_rank": 0,
       "now_cost_rank_type": "",
       "form_rank": 0,
       "form_rank_type": "",
   },
   "expected": {
       "expected_goals": 0,
       "expected_assists": 0,
       "expected_goal_involvements": 0,
       "expected_goals_conceded": 0,
       "ep_next": 0,
       "ep_this": 0,
   },
   "stats_per_90": {
       "points_per_game_rank": 0,
       "points_per_game_rank_type": "",
       "starts_per_90": 0,
       "clean_sheets_per_90": 0,
       "defensive_contribution_per_90": 0,
       "expected_goals_per_90": 0,
       "saves_per_90": 0,
       "expected_assists_per_90": 0,
       "expected_goal_involvements_per_90": 0,
       "expected_goals_conceded_per_90": 0,
```

```
        "goals_conceded_per_90": 0,
        "points_per_game": 0,
    },
}
```

Purpose:

- Represents a single FPL player with:
  - Raw stats (minutes, goals, assists, defensive actions).
  - FPL-specific stats (bonus, BPS, ICT, transfers, ownership).
  - Ranks (form, cost, ICT, etc.).
  - Expected metrics (xG, xA, xGI, xGC, ep_next, ep_this).
  - Per-90 derived stats.
- This is the **primary input** for future Cooks: minutes probability, form, role, and xP.

---

6. Current system boundary (pre-Cook)

At this point in the project:

- ✅ Producers are implemented and writing into Redis.
- ✅ Canonical schemas for **FPL_FIXTURES**, **FIXTURES**, **TEAMS**, **PLAYERS** are defined and populated.
- ✅ Redis access layer is in place.
- ✅ Folder structure cleanly separates config, producers, db, and (future) cooks.

Not yet implemented in this document:

- Cooks (analytical workers).
- xP Aggregator.
- Manager (Best XI selector).
- API/Waiter.
- Dashboard.

This document is the **reference snapshot** of the system **right before** analytical logic is added.
